

ORGANIZACIÓN Y BÚSQUEDA EFICIENTE DE UN CATÁLOGO BIBLIOGRÁFICO

202505232 · Angel Daniel Ruiz Ramos · Sección C

Resumen

Este ensayo analiza la organización y la búsqueda de libros mediante estructuras de datos propias en una aplicación web desarrollada en C#. El problema consiste en representar categorías jerárquicas y localizar registros en un catálogo cuyo crecimiento vuelve insuficiente una búsqueda secuencial. Se sostiene que la clasificación temática y los índices de consulta deben cumplir funciones distintas, aunque compartan los mismos objetos del dominio. La propuesta combina una jerarquía de categorías con árboles AVL por ISBN y por título, listas enlazadas para presentar resultados e importación incremental de archivos XML. También examina la coordinación de los índices, la prevención de duplicados y la generación de reportes con Graphviz. Su aporte técnico es mostrar cómo las decisiones sobre enlaces, balance y responsabilidades afectan la eficiencia y la coherencia del sistema. Las verificaciones locales respaldan el funcionamiento bajo las precondiciones descritas. Se concluye que la búsqueda eficiente requiere mantener las invariantes de las estructuras y reconocer límites como la memoria temporal y las dependencias externas.

Palabras clave

Catálogo, jerarquía, árboles, búsqueda, XML.

Abstract

This essay examines book organization and retrieval through custom data structures in a web application developed in C#. The problem involves representing hierarchical categories and locating records in a growing catalog where sequential search becomes insufficient. It argues that thematic classification and search indexes should serve different purposes while sharing the same domain objects. The proposed solution combines a category hierarchy with AVL trees indexed by ISBN and title, linked lists for presenting results, and incremental XML file imports. It also examines index coordination, duplicate prevention, and report generation with Graphviz. Its technical contribution is to show how decisions about links, balancing, and responsibilities affect system efficiency and consistency. Local checks support correct operation under the stated preconditions. The essay concludes that efficient retrieval requires preserving structural invariants and recognizing limitations such as temporary in-memory storage and external dependencies.

Keywords

Catalog, hierarchy, trees, search, XML.

Introducción

Una colección bibliográfica extensa requiere mecanismos para clasificar sus registros y encontrarlos sin revisar todos los libros. El Proyecto 2 de Introducción a la Programación y Computación 2 plantea este problema mediante un catálogo de más de cien mil libros y exige estructuras de datos propias (Universidad de San Carlos de Guatemala, 2026). Este ensayo explica cómo una jerarquía de categorías y varios índices AVL responden a necesidades diferentes dentro de una misma aplicación. Se argumenta que la eficiencia depende tanto del balance de los árboles como de la coordinación de sus modificaciones. El análisis aborda la representación de datos, la importación XML, la interfaz web y los reportes gráficos. Finalmente, contrasta las verificaciones realizadas con las precondiciones y limitaciones de la implementación.

Desarrollo del tema

De una colección de registros a un sistema organizado

Administrar una librería exige resolver dos necesidades distintas: localizar un libro y comprender el lugar que ocupa dentro del catálogo. Una lista extensa puede guardar títulos, autores e identificadores, pero por sí sola no expresa relaciones entre temas ni evita revisar muchos registros al buscar uno. El problema del Proyecto 2 plantea una colección superior a cien mil libros y una jerarquía de categorías. La solución debe atender ambas necesidades sin sustituir los tipos de dato abstracto propios por las colecciones incorporadas del lenguaje (Universidad de San Carlos de Guatemala, 2026). La tesis de este diseño es que separar la organización temática de los índices de búsqueda permite conservar un modelo comprensible y, al mismo tiempo, reducir el trabajo necesario para encontrar los libros.

La representación de una categoría reproduce una relación familiar: cada tema puede contener otros más específicos. Una categoría guarda su nombre, una referencia a su padre, otra a su primer hijo y otra a su siguiente hermano. Estos enlaces permiten representar una cantidad variable de hijos sin reservar un arreglo de

tamaño fijo. También hacen innecesario crear una clase adicional que solo envuelva la categoría. La categoría es simultáneamente un dato del dominio y un nodo de su jerarquía. Las categorías principales se enlazan como hermanas y pueden existir varias raíces, decisión que permite organizar áreas independientes sin inventar una categoría artificial.

La inserción mantiene ordenados alfabéticamente a los hermanos. Para comparar nombres se utiliza `string.Compare` con la opción de ignorar mayúsculas y las reglas de la cultura actual. El recorrido de búsqueda desciende al primer hijo cuando puede; de lo contrario, asciende hasta encontrar un hermano pendiente. La referencia al padre permite realizar este recorrido sin una pila incorporada. Sin embargo, una jerarquía temática no es automáticamente un índice ordenado por todos los nombres del catálogo: localizar una categoría puede requerir visitar hasta c categorías. Reconocer este costo $O(c)$ evita atribuir a toda la aplicación la eficiencia del árbol de libros.

El libro almacena un ISBN de tipo `long`, un título, un autor y su categoría. Sus propiedades son de solo lectura después de construir el objeto. Esta decisión reduce las posibilidades de cambiar una clave sin reorganizar el índice correspondiente. Los nodos de los árboles son objetos separados: varios nodos pueden apuntar al mismo libro sin copiar su información. Cada categoría mantiene un árbol de sus libros directos y el catálogo conserva un árbol general. Consultar una categoría muestra únicamente los libros clasificados en ella; los de sus descendientes se consultan desde sus respectivas categorías. Esta delimitación hace explícito un aspecto que el enunciado no precisa.

La interfaz presenta estos conceptos mediante una colección de libros y una página de categorías. Los formularios permiten registrar datos, las consultas muestran resultados y los gráficos explican las relaciones. De esta manera, las estructuras internas sirven a operaciones reconocibles para una persona que organiza una librería. La claridad del modelo no depende solamente de elegir un árbol, sino de mantener una correspondencia estable entre cada estructura y la pregunta que debe responder.

Índices balanceados y consistencia del catálogo

El ISBN es una clave apropiada para encontrar un libro concreto porque el enunciado lo define como único. Un árbol binario de búsqueda organiza las claves menores a la izquierda y las mayores a la derecha. No obstante, insertar claves ya ordenadas en un árbol sin balance puede producir una cadena, con un costo de búsqueda semejante al de una lista. Por ello se utiliza un árbol AVL propio. Cada nodo conserva su altura y la diferencia entre las alturas de sus hijos debe permanecer entre menos uno y uno. Esta condición limita la altura del árbol y sostiene búsquedas de costo $O(\log n)$ en el índice global.

La implementación asigna altura cero a una referencia vacía y altura uno a una hoja. Después de insertar o eliminar, Balancear actualiza la altura antes de decidir si necesita una rotación. Una rotación simple modifica algunos enlaces conservando el orden de las claves. Cuando el hijo inclinado se orienta hacia el lado contrario, se realizan dos rotaciones. Al actualizar las alturas después de cambiar los enlaces, las siguientes decisiones de balance utilizan información coherente. Estos detalles son importantes: un árbol que parece ordenado puede tener alturas incorrectas y fallar solamente después de una secuencia posterior de operaciones.

Eliminar un nodo con dos hijos requiere conservar ambos subárboles. La solución toma el sucesor, es decir, el menor nodo del subárbol derecho, lo retira de su ubicación y lo coloca en el lugar del nodo eliminado. Se traslada el nodo completo en vez de reasignar su propiedad Libro, que no dispone de un setter. El proceso termina balanceando los nodos afectados. El mínimo se obtiene siguiendo enlaces izquierdos y el máximo siguiendo enlaces derechos. Para listar la colección, el recorrido en orden visita izquierda, libro y derecha, agregando las referencias a una lista enlazada propia en tiempo $O(n)$.

La descripción del problema también menciona la localización por título (Universidad de San Carlos de Guatemala, 2026). Para cubrir esa consulta se incorpora un segundo AVL, ordenado primero por título y después

por ISBN. El título no tiene que ser único: dos libros pueden compartirlo y distinguirse por su ISBN. La búsqueda utiliza el título completo sin distinguir mayúsculas y devuelve las coincidencias en una ListaLibros. El recorrido se limita a las ramas que pueden contener la clave, con costo $O(\log n + k)$, donde k es la cantidad de resultados. No es una búsqueda de fragmentos ni de palabras aproximadas. El nodo concreto NodoLibro se reutiliza como tipo, pero cada índice crea sus propios nodos.

Mantener varios índices exige una única puerta de entrada para modificar el dominio. Catalogo registra el libro en el árbol global, en el de su categoría y en el índice de títulos. Al eliminarlo, retira sus referencias de los tres. La reutilización del mismo objeto Libro permite comprobar que los índices representan el mismo dato. El costo de registro incluye la búsqueda de categoría $O(c)$, además de las inserciones balanceadas; por tanto, no debe describirse toda la operación como $O(\log n)$. La consistencia depende de utilizar Catalogo en vez de modificar aisladamente uno de los árboles.

Importación, interfaz y representación gráfica

La importación XML permite iniciar una colección o ampliarla con nuevos archivos. El catálogo no se reemplaza después de cada carga. CargadorXml recibe el texto y utiliza XmlReader para reconocer los elementos del formato: config, listaCategorias y listaLibros. Las dos secciones son opcionales. El archivo original se lee sin modificarlo y las entidades XML se interpretan mediante el lector. Las listas y los árboles del dominio siguen siendo implementaciones propias; el lector XML cumple la función de interpretar el documento, no la de almacenar el catálogo.

Una dificultad de la importación es que un hijo puede aparecer antes que su padre. Para resolverla se hace una primera lectura de categorías y se enlazan objetos CategoriaPendiente. Cada pasada incorpora aquellas categorías sin padre o cuyo padre ya se encuentra en el catálogo, y retira sus pendientes. Las restantes se revisan en la siguiente pasada. Después se efectúa una segunda lectura para registrar los libros mediante Catalogo. Así,

incluso cuando `listaLibros` aparece antes de `listaCategorias`, los libros se incorporan cuando sus categorías ya existen. El algoritmo favorece la claridad, aunque puede realizar mucho trabajo cuando hay numerosas categorías declaradas en orden inverso de dependencia.

La aplicación utiliza ASP.NET Core MVC para separar la recepción de solicitudes, los datos y su presentación. `CatalogoController` recibe formularios, llama a los servicios y selecciona las vistas. Los formularios que modifican información incluyen el token antifalsificación de MVC. Esta protección de solicitudes no sustituye las reglas del dominio. `EstadoCatalogo` se registra una vez por instancia de aplicación y contiene el catálogo y el objeto usado para sincronizar accesos. Las operaciones sobre las estructuras se realizan dentro del mismo bloqueo. Esto evita que dos solicitudes modifiquen simultáneamente los enlaces o consulten un árbol mientras otra operación lo reorganiza.

Las vistas reciben listas propias que se recorren mediante `Primero` y `Siguiente`. La lista de categorías contiene una copia del nombre, del nombre del padre y del nivel; la vista no necesita recorrer una jerarquía que otra solicitud pudiera estar modificando. Las listas de libros conservan referencias a objetos con datos de solo lectura. La página principal presenta primero el registro y después las búsquedas y el catálogo. La carga XML dispone de una página independiente. Las vistas Razor usan Bootstrap básico mediante un enlace externo. La página de categorías permite crear temas y consultar sus libros o sus ramas. La opción Ayuda presenta los datos del estudiante y abre la documentación desde la misma aplicación, sin depender de un servicio externo.

Los reportes se construyen primero como texto DOT y se procesan con Graphviz. El gráfico de categorías representa relaciones padre-hijo; una consulta por rama comienza en la categoría seleccionada y excluye sus padres y hermanos. El reporte de libros usa una secuencia ascendente por ISBN, adecuada para el requisito de presentación, aunque no represente la forma interna del AVL. Los textos se escapan para conservar comillas, barras y saltos de línea. La construcción del DOT ocurre dentro del bloqueo y el proceso externo se ejecuta

después de liberarlo. El resultado SVG se abre como un documento gráfico independiente y permite examinar la organización con mayor amplitud.

Verificación, límites y conclusiones

La calidad de una estructura dinámica se evalúa por sus propiedades después de varias operaciones, no solamente por la apariencia de un ejemplo. Las comprobaciones locales verifican el orden de las claves, las alturas almacenadas y la diferencia de altura entre hijos. También comprueban que el libro referenciado desde el árbol general sea el mismo que aparece en su categoría. Al eliminar datos, se revisa que los índices dejen de devolverlos y que los árboles restantes conserven sus propiedades. Estas comprobaciones ayudan a detectar errores de enlaces que podrían permanecer ocultos si solo se observaran las respuestas de una búsqueda (Ruiz Ramos, 2026).

La revisión previa de la base ejecutó 71 comprobaciones funcionales que incluyeron cargas XML sucesivas, secciones opcionales, padres posteriores, entidades, CDATA y generación real de SVG. Una prueba adicional importó 100,003 libros distribuidos en 101 categorías, consultó todos los ISBN y eliminó la colección hasta dejar los árboles vacíos. Produjo 1,300,139 comprobaciones correctas. En esa ejecución, anterior a incorporar el índice por título, la carga tardó 977 milisegundos y las búsquedas globales 26 milisegundos. Estas cifras describen una medición local; no garantizan el mismo tiempo en otro equipo ni representan una prueba de renderizado gráfico masivo o de navegación web.

La implementación impide repetir ISBN y nombres de categoría. `Catalogo` consulta la clave antes de modificar las estructuras y devuelve null cuando el registro ya existe. El controlador muestra una alerta y conserva el dato original, sin cambiar sus índices ni su clasificación. En la carga XML se omiten los duplicados y se informa su cantidad; los registros nuevos sí se incorporan. Los nombres se comparan sin distinguir mayúsculas, incluso entre niveles distintos de la jerarquía. Además, el formulario de categorías rechaza nombres vacíos con una alerta y el generador de reportes protege el escape de

textos nulos. Se mantienen las demás precondiciones: las referencias existen y no hay ciclos. Un padre inexistente puede impedir terminar la importación; eliminar un ISBN inexistente tampoco forma parte del contrato admitido.

También existen límites operativos. El catálogo se conserva en memoria y desaparece al reiniciar la aplicación. Los usuarios de una misma instancia comparten la colección; no hay aislamiento por sesión. Graphviz debe estar instalado y su ejecutable debe ser accesible. Su ejecución no incorpora recuperación de fallos ni un tiempo máximo de espera. Además, la consulta de categoría continúa siendo lineal y un informe enorme puede ser difícil de representar y de leer. Estos límites orientan mejoras futuras, como un índice adicional de categorías, una estrategia de presentación por partes y una mejor gestión de fallos externos, siempre conservando las restricciones académicas.

Las comprobaciones HTTP más recientes verificaron 112 condiciones de navegación y funcionamiento; otras 18 revisaron el rechazo de categorías vacías y la conservación de los reportes. Estos resultados no constituyen una garantía sobre entradas arbitrarias ni sobre todos los posibles entornos.

Conclusiones

En conclusión, una jerarquía de categorías y varios índices AVL resuelven preguntas distintas dentro de un mismo catálogo. Los enlaces temáticos permiten comprender la clasificación; los árboles balanceados permiten localizar libros con un costo acotado; las listas propias presentan los resultados sin introducir colecciones prohibidas. Catálogo coordina las modificaciones y la interfaz hace accesibles esas capacidades. La solución es defendible cuando se explican tanto sus algoritmos como sus límites. La entrega académica exige, además del programa, documentación, diagramas e historial de versiones: las pruebas técnicas complementan esos elementos, pero no los reemplazan.

Referencias bibliográficas

Ruiz Ramos, A. D. (2026). Proyecto 2 de IPC2: Catálogo bibliográfico [Código fuente y registros de verificación no publicados].

Universidad de San Carlos de Guatemala. (2026). Proyecto No. 2 de Introducción a la Programación y Computación 2, segundo semestre de 2026 [Enunciado de laboratorio].

Anexo A Modelo de dominio

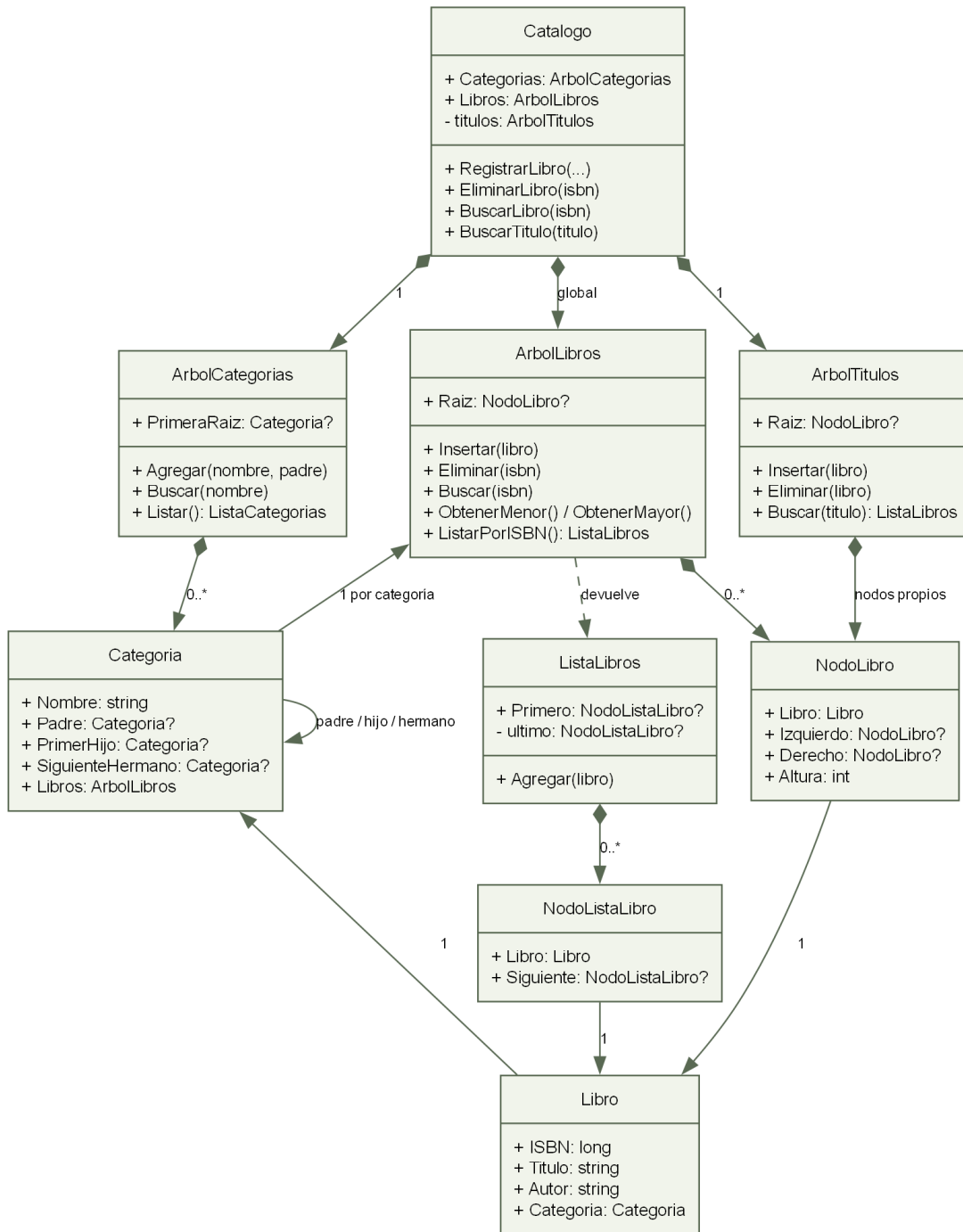


Figura 1. Diagrama de clases del dominio.

Fuente: elaboración propia. Se muestran miembros representativos.

Anexo B Servicios y presentación

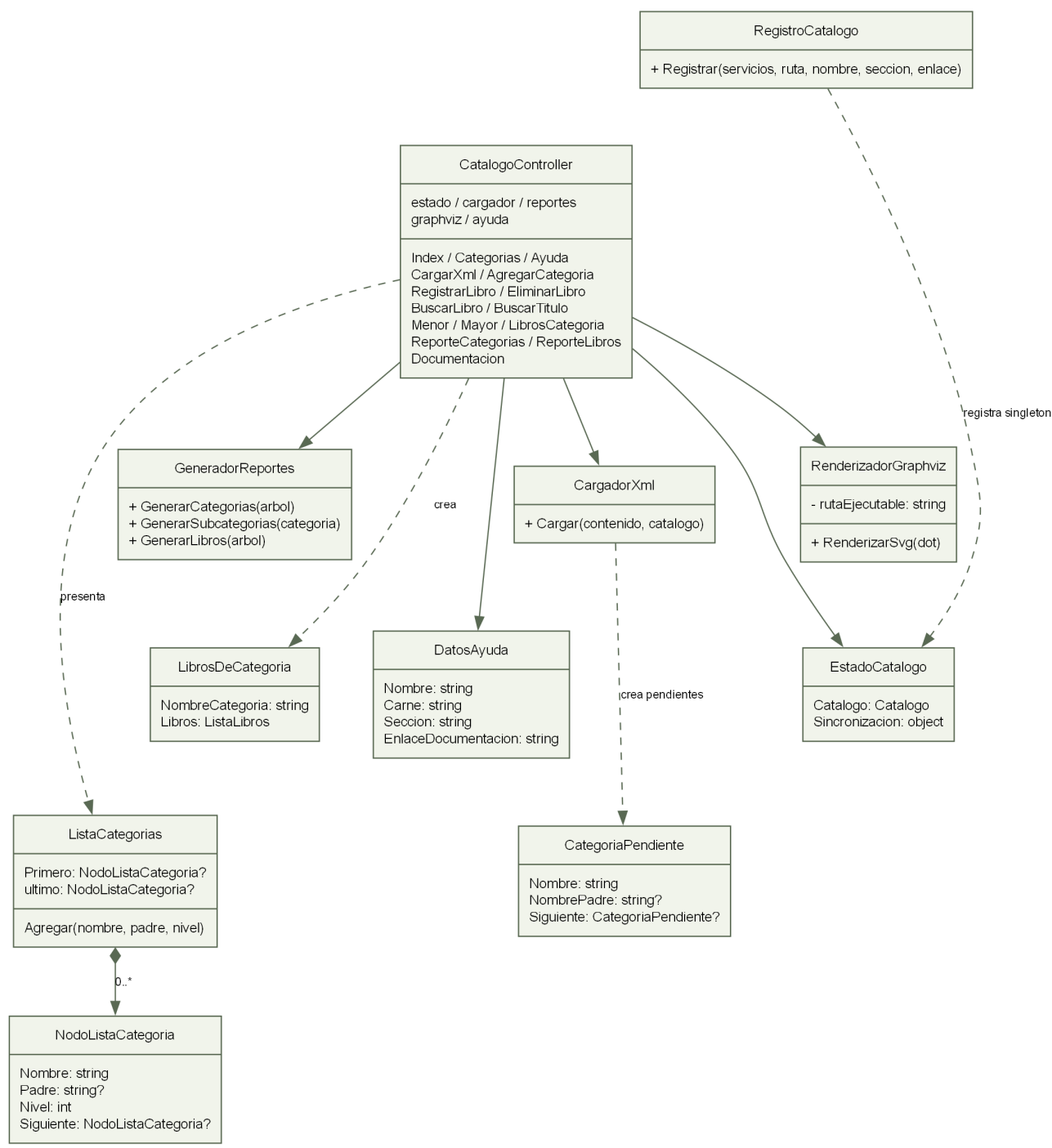


Figura 2. Diagrama de clases de servicios y presentación.

Fuente: elaboración propia. Se muestran miembros representativos.